# Software Testing

# What is a test?

An **experiment** to determine the outcome of a program execution.

- A test is **passed** if the outcome matches our expectations.
- A test is **failed** if the outcome differs from our expectations.

We can test a program when:

- The program compiles and runs.
- The expected outcome is understood.

Note: We can test an *incomplete* program as long as it compiles and runs!

# Why care about testing?

As a technique to ensure correctness.

As a method to implement tricky or hard-to-understand algorithms.

As a mechanism to enable change and refactoring without causing breakage.

# My Initial View

**As a technique to ensure correctness.**

As a method to implement tricky or hard-to-understand algorithms.

As a mechanism to enable change and refactoring without causing breakage.

# My Changed View

As a technique to ensure correctness.

As a method to implement tricky or hard-to-understand algorithms.

**As a mechanism to enable change and refactoring without causing breakage.**

# Technical Debt

Every codebase has an amount of *technical debt.*

You increase the technical debt:

- Every time you add features without documentation or tests.
- Every time you add a hack without refactoring it afterwards.
- Every time a feature must be ready “by Monday”.
- Every time a new version of a language is released and you do not update.
- Every time a new version of a library is released and you do not update.
- Every time a new version of s platform is released and you do not update.

# Software Rot

As the technical debt increases the codebase literally rots!

What can be done to decrease the technical debt?

- Continuously improve and maintain the codebase through refactoring.
- Continuously update to newest versions of languages, libraries, platforms.
- **Add unit and system tests to ensure the correctness and maintainability.**

**A codebase may incur so much technical debt that it is cheaper to throw it away and start from scratch than it is to fix it!**

- Happens all the time in software development, but not good!

# Dimensions of Testing

| Question | Dimensions | |
|---|---|---|
| *How are tests run?* | Manual | Automated |
| *What does a test cover?* | Unit Test | System Test |
| *What assumptions does the test make about the implementation?* | Black Box | White Box |
| *Is the test written before or after the implementation is written?* | Test First | Test Later |

***Every decision involves different trade-offs.***

# Manual vs. Automated Testing

**Manual Testing:** A human runs the program and inspects the result.

- Advantages: works well for some scenarios, e.g. UI testing.
- Disadvantages: tedious, time consuming, and error-prone.

**Automated Testing:** A computer runs the program and compares the result to a known expected result.

- Advantages: tests are repeatable, little manual effort.
- Neutral: requires time in the short run, but pays off in the long run.
- Disadvantages: hard to test certain requirements.

# Unit Test vs. System Test

**Unit testing** involves testing individual components in isolation.

- A unit test might test **a single method or a few methods of a class.**

**System testing** involves testing larger components in cooperation.

- A system test might test **numerous classes working together.**

A good test suite should have both unit and system tests.

- Both are necessary and serve different purposes.
- Unit testing is easy. System testing is hard.
- Big Companies invest Big Bucks in good system tests. (Think Google, …)

# Introduction to jUnit

A Java Unit Testing Framework

- Extremely popular and widely-used.
- Integrated into Intellij IDEA.
- Easy to use and get started with.
- Annotation based.



# Asserts in jUnit

| Annotation | Meaning |
| --- | --- |
| `assertTrue` | Asserts that the given value is true. |
| `assertFalse` | Asserts that the given value is false. |
| `assertNull` | Asserts that the given value is null. |
| `assertNotNull` | Asserts that the given value is not null. |
| `assertEquals` | Asserts that the two values are equal. |
| `assertArrayEquals` | Asserts that the two arrays are equal. |
| `fail` | Unconditional fails the current unit test. |

# Examples

```
@Test
public void isOverdrawn01() {
    BankAccount account = new BankAccount(100);
    account.withdraw(500_000);
    assertTrue(account.isOverdrawn());
}

@Test
public void isOverdrawn02() {
    BankAccount account = new BankAccount(100);
    account.withdraw(25);
    assertFalse(account.isOverdrawn());
}
```

# Annotations

| Annotation | Meaning |
| --- | --- |
| `@Test` | Marks the method as a unit test. |
| `@BeforeAll` | Marks that a method should run before all unit tests are run. |
| `@BeforeEach` | Marks that a method should run before each unit test. |
| `@AfterEach` | Marks that a method should run after each unit test has run. |
| `@AfterAll` | Marks that a method should run after all unit tests have run. |
| `@Disabled` | Marks that a test should be skipped. |

# Example: BeforeEach

```
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class TestBankAccount {

    BankAccount account = null;

    @BeforeEach
    public void beforeEach() {
        account = new BankAccount(100);
    }

    @Test
    public void withdraw01() {
        account.withdraw(25);
        assertEquals(75, account.getBalance());
    }

}
```

# Black Box vs. White Box

A **white box test** has knowledge of the unit under test (UUT).

- Easy to know what corner cases to test.
- Easy to know when we have tested sufficiently.
- Tests *may not* be faithful to the specification.
- Tests *may not* sufficiently test a *different* implementation.
- Tests and implementation are coupled.

A **black box test** has *no* knowledge of the unit under test.

- Relies on the specification and only the specification.
- Avoids any assumptions about the implementation.
- A more paranoid and adversarial approach.
- Red Team – Blue Team.

# Test Coverage

Q: How do determine if a test suite is useful?

A: We measure the *test coverage* of a test suite:

- **Method Coverage:** the percentage of methods executed.
- **Statement Coverage:** the percentage of statements executed.
- **Path Coverage:** the number of paths taken.

- We have great IDE support for this!

Ensuring high test coverage is a measurable and achievable goal, but…

# Production and Test Code

We should keep **production** and **test code** completely separate.

- Production code must never depend on the test code!
- A common strategy is to separate the code into two directories:
  - Production code goes in src/
  - Test code goes in test/

As a general rule the test code is *not* part of any release.

- Test code may contains confidential information (e.g. database credentials).
- Test code may contain insecure components (e.g. security overrides).
- Counter-point: Open source *libraries* typically do ship their tests.

# Assertions

We use unit tests and assertions to check for correctness.

- We do not ship these tests to the customer (see previous slide).

We can also add assertions directly to the product code.

- Advantages: We immediately discover abnormal situations during program execution and the program is (likely) shutdown before it does any harm.
- Disadvantages: We must pay a performance cost for these checks.

```
public void deposit(int amount) {
    assert amount >= 0 : “Non-positive amount”
    ...
}
```

# Contracts

Pre and Post Conditions

- A **pre condition** is a boolean expression which must be true immediately before execution of a method.
- A **post condition** is boolean expression which must be true immediately after execution of a method.

A **contract** is pre and post condition of a method.

- It is a formal way to state that “if you promise X then I will promise Y”.

```
public void deposit(int amount) {
    assert amount >= 0 : “Pre-condition failed.”
    ...
    assert balance == oldBalance + amount : “Post-condition failed.”
}
```

# It is impossible to test X!

"If a feature cannot be tested
it simply does not exist."

# Dijkstra on Testing



"Testing cannot be used to show the absence of bugs, only to show their presence."

*We can always write one more test. We will never be done!*

# When should tests be written?

*Before* writing the implementation:

- Helps explore the problem domain.
- Helps design the appropriate methods and interfaces.
- Helps drive the implementation.
- Ensures we have tests when we are done.

*After* writing the implementation:

- We have better knowledge of the problem domain.
- Easier to determine what tests to write.
- Quicker to write those tests.
- Risk of saying “we will add tests later” (never happens.)

# Test-Driven Development

**A method, not for testing, but for software development.**

**Key Idea:**

Use unit tests to drive the implementation!

Added bonus: You get unit tests for free.

# The TDD Rhythm

1. Quickly add a test.
2. Run all tests and see the new one fail.
3. Make a small change.
4. Re-run all tests and see them succeed.
5. Refactor the code.
6. (Re-run all tests.)

# The Three Laws of TDD

According to Robert Martin “Uncle Bob”:

**First Law:** You may not write production code until you have written a failing unit test.

**Second Law:** You may not write more of a unit test than is sufficient to fail, and not compiling is failing.

**Third Law:** You may not write more production code than is sufficient to pass the currently failing test.

According to Bob, if we do this: We will write dozens of tests every day, hundreds of tests every month, and thousands of tests every year.

# F.I.R.S.T. Principles

**Fast**

- Tests should run fast so that you can run them often.

**Independent**

- Tests should be independent to ensure correctness and ease of debugging.

**Repeatable**

- Tests should run on any environment, including your laptop and build server.

**Self-Validating**

- Every test should have a clear boolean output: Test failed for this reason.

**Timely**

- Tests should be written before writing the production code.